

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	Barath
Register Number	3122247001011
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To build spam/ham classifiers for the Spambase dataset using Naïve Bayes (Gaussian, Multinomial, Bernoulli) and K-Nearest Neighbors, compare the three Naïve Bayes variants, study the effect of varying k , compare the KDTree and BallTree neighbour-search algorithms, optimize KNN using GridSearchCV and RandomizedSearchCV, evaluate computational complexity and execution time, and evaluate all models using K-Fold cross-validation.

2 Problem Statement

Given the Spambase dataset of word/character frequency and capital-letter-run features extracted from 4,601 emails, the problem is binary classification: predict whether each email is spam (1) or not-spam (0). The input is a 57-dimensional numeric feature vector per email; the output is a binary class label. The objective is to identify which Naïve Bayes variant and which KNN configuration classify spam most accurately and efficiently, and to understand the computational trade-offs between the two model families.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase (UCI ML Repository)
Number of Samples	4,601
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (2,788 not-spam / 1,813 spam)
Missing Values	0
Train-Test Split	80:20 stratified (3,680 train / 921 test), <code>random_state=42</code>

4 Exploratory Data Analysis

Include all relevant visualizations.

- Dataset overview

- Statistical summary
- Missing value analysis
- Class distribution
- Correlation matrix
- Feature distribution

Class Distribution

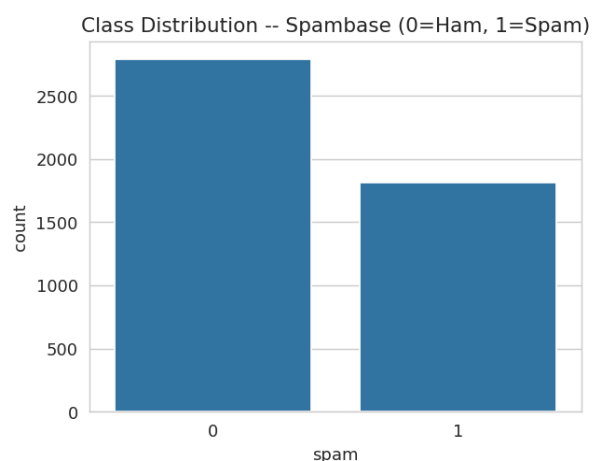


Figure 1: Class distribution – Spambase

Inference: The dataset is moderately imbalanced (2,788 ham vs. 1,813 spam, about 61:39). This is mild enough that accuracy remains a reasonably informative metric, but precision/recall/F1/ROC-AUC are still reported throughout to avoid overstating performance. No missing values are present, so no missing-value visualization is required.

Correlation Matrix

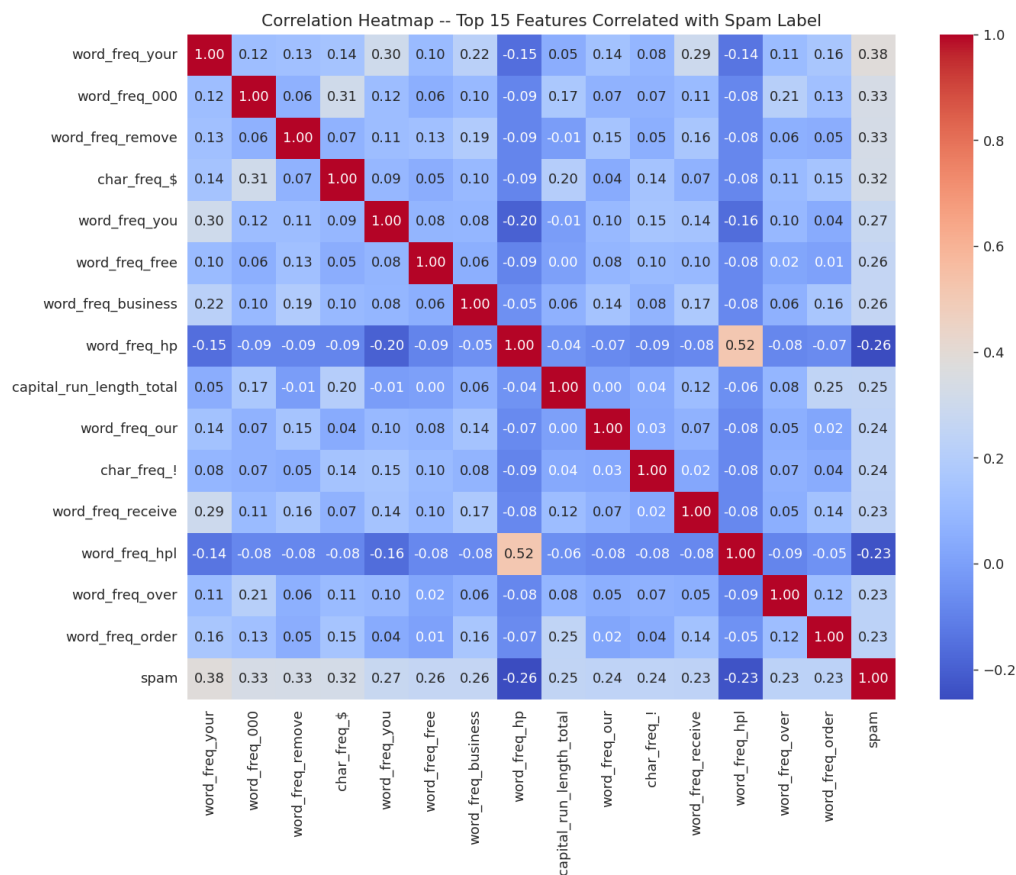


Figure 2: Correlation heatmap – top 15 features most correlated with the spam label

Inference: Character frequency features such as `char_freq_!` and `char_freq_$`, along with word frequencies for “your”, “000”, “remove”, and “free”, show the strongest positive correlation with the spam label – consistent with common spam-email vocabulary and formatting (exclamation marks, dollar signs, promotional words).

Feature Distribution

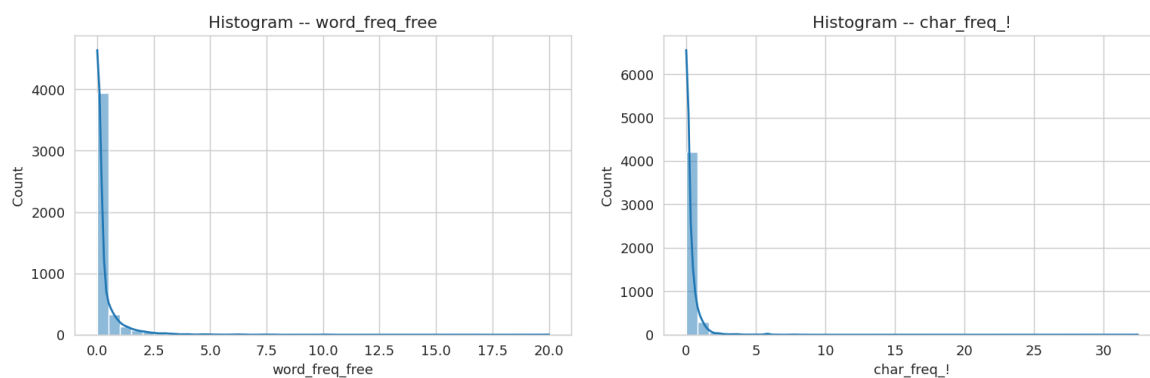


Figure 3: Histograms of important features – `word_freq_free` and `char_freq_!`

Inference: Both features are heavily right-skewed with a large mass at or near zero, since most legitimate emails never use these tokens, while a minority of (mostly spam) emails use them repeatedly – exactly the long-tail pattern that makes them discriminative.

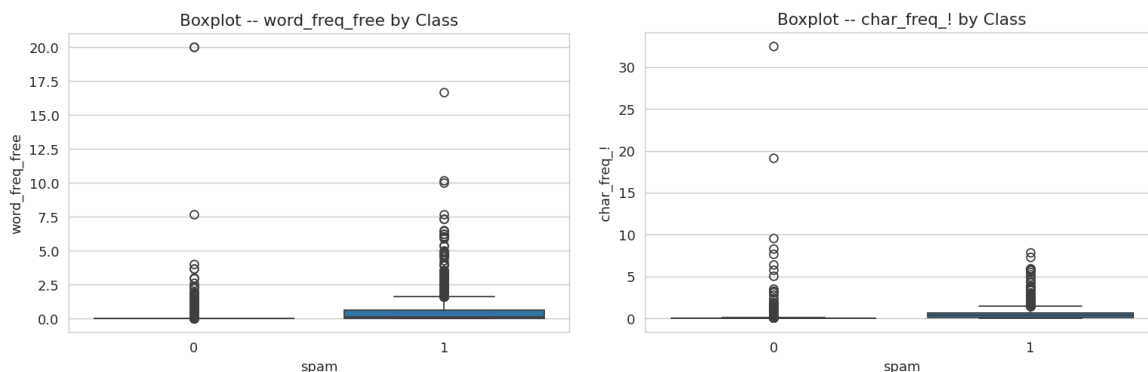


Figure 4: Boxplots of word_freq_free and char_freq_! grouped by class

Inference: Both features show a visibly higher median and wider spread for the spam class (1) compared to ham (0), confirming they carry real discriminative signal rather than noise – this matches their high ranking in the correlation heatmap above.

5 Data Preprocessing

Explain:

- **Missing value handling:** Not required – the dataset contains zero missing values across all 4,601 rows and 57 features.
- **Encoding:** Not required – all 57 features and the target label are already numeric.
- **Feature scaling:** StandardScaler (zero mean, unit variance) was applied for KNN and Gaussian NB, since distance-based and Gaussian-likelihood models are sensitive to feature scale. Multinomial NB used the raw non-negative frequency values directly (required, since it models non-negative counts). Bernoulli NB internally binarizes each feature at a 0.0 threshold (scikit-learn default).
- **Train-test split:** 80:20 stratified split (3,680 train / 921 test rows), preserving the 61:39 class ratio in both subsets.
- **Feature engineering:** Not applicable – the 57 provided frequency/length features were used as-is.

6 Machine Learning Algorithm

Explain:

- **Working principle:** Naïve Bayes applies Bayes' theorem, $P(C | X) = P(X | C)P(C)/P(X)$, with a conditional independence assumption between features, to compute each class's posterior probability directly from training-data frequency/-likelihood statistics – there is no iterative optimization. Gaussian NB assumes each

feature is normally distributed within a class; Multinomial NB models features as counts/frequencies; Bernoulli NB treats each feature as a binary presence/absence indicator. KNN is a lazy, instance-based learner: it stores the training data and, at prediction time, finds the k nearest training points (by Euclidean/Manhattan distance) to a query point and assigns the majority class among them (optionally weighted by inverse distance).

- **Mathematical formulation:**

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}, \quad \hat{y} = \underset{i \in N_k(x)}{\text{mode}} y_i$$

where $N_k(x)$ is the set of k nearest neighbours of query point x .

- **Advantages:** Naïve Bayes is extremely fast to train and predict ($O(nd)$ / $O(d)$), works well even with limited data, and requires no hyperparameter tuning. KNN makes no assumption about the underlying data distribution and can capture non-linear decision boundaries.
- **Limitations:** Naïve Bayes' independence assumption is frequently violated in practice (many Spambase features are correlated, as seen in Section 4), which can bias its probability estimates. KNN's prediction cost scales with training set size ($O(nd)$ for brute force), it is sensitive to irrelevant/unscaled features, and it stores the entire training set in memory.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

Parameter	Value
Best $n_neighbors$ (GridSearchCV)	9
Best weights (GridSearchCV)	distance
Best metric (GridSearchCV)	manhattan
Best CV Accuracy (GridSearchCV)	0.9253
Best $n_neighbors$ (RandomizedSearchCV)	10
Best weights (RandomizedSearchCV)	distance
Best metric (RandomizedSearchCV)	euclidean
Best CV Accuracy (RandomizedSearchCV)	0.9234

Note: GridSearchCV exhaustively searched $n_neighbors \in \{1, 3, 5, 7, 9, 11, 13, 15\} \times weights \in \{uniform, distance\} \times metric \in \{euclidean, manhattan\}$ (32 combinations, 10.69s). RandomizedSearchCV sampled 20 combinations from $n_neighbors \in [1, 30]$ with the same weights/metric options (6.07s), trading a marginal accuracy difference for roughly $1.75\times$ less computation.

9 Results

Metric	Value (Best KNN, GridSearchCV-tuned)
Accuracy	0.9207
Precision	0.9421
Recall	0.8512
F1-score	0.8944
ROC-AUC	0.9712

Comparison context: The best Naïve Bayes variant, Bernoulli NB, achieved Accuracy 0.8762, Precision 0.8716, Recall 0.8044, F1 0.8367, ROC-AUC 0.9496 on the same test set – Best KNN outperforms it on every metric except training/prediction speed (Section 9).

10 Result Visualization

Include all graphs generated during the experiment.

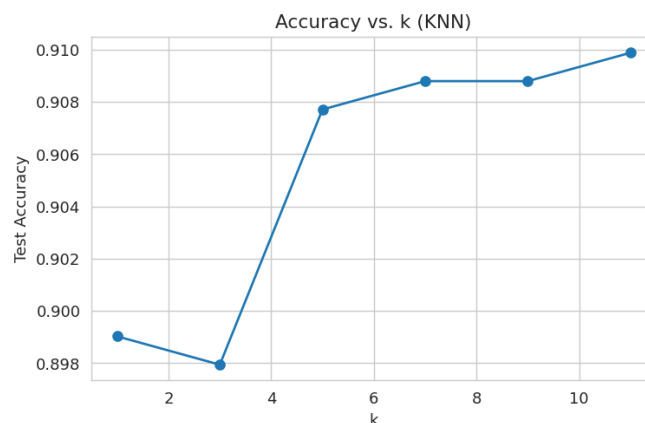


Figure 5: Accuracy vs. k (KNN, unweighted Euclidean)

Inference: Accuracy rises steadily from $k = 1$ to $k = 11$ with no sign of turning downward yet, suggesting the optimum k for this dataset lies at or beyond $k = 11$ – consistent with GridSearchCV independently selecting $k = 9$ once distance weighting and the Manhattan metric are also allowed to vary.

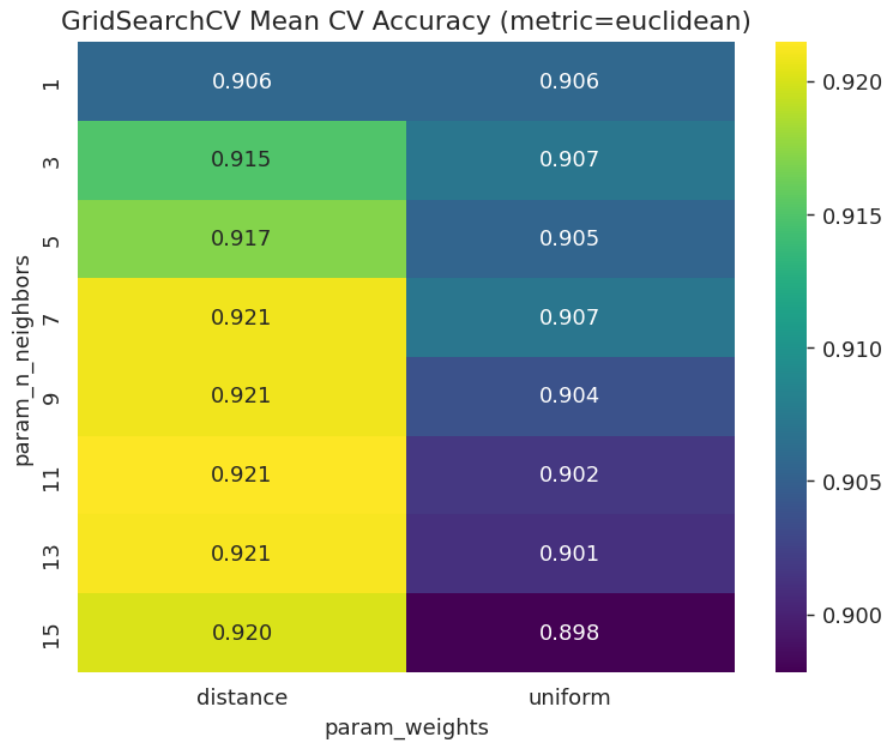


Figure 6: GridSearchCV mean CV accuracy heatmap ($n_neighbors \times weights$, metric=euclidean)

Inference: Accuracy consistently improves moving from **uniform** to **distance** weighting at every k , and generally increases with k up to the tested range – both trends corroborate the accuracy-vs- k curve above and the overall best-parameter choice.

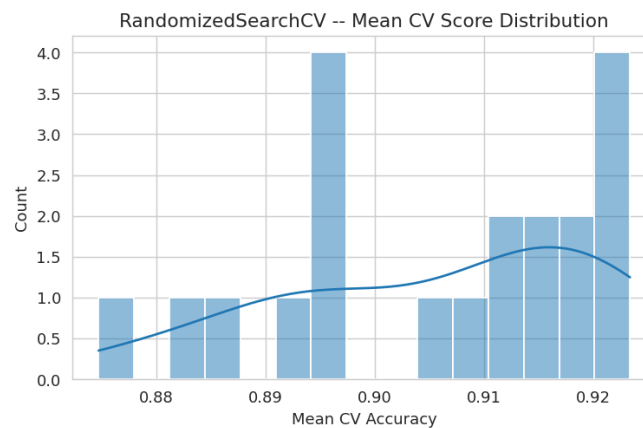


Figure 7: RandomizedSearchCV mean CV score distribution across 20 sampled configurations

Inference: Most of the 20 randomly sampled hyperparameter combinations cluster in a fairly narrow, high-accuracy band (roughly 0.90–0.925), showing KNN’s performance on this dataset is fairly robust to the exact choice of k /metric/weighting once a reasonable neighbourhood size is used.

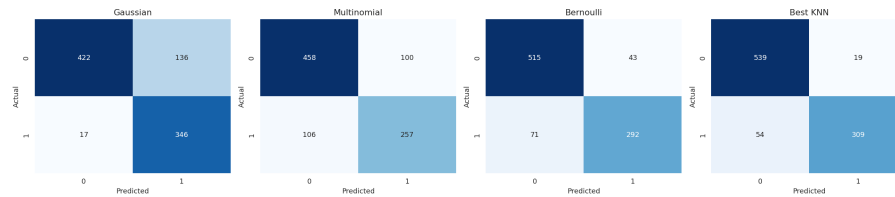


Figure 8: Confusion matrices – Gaussian NB, Multinomial NB, Bernoulli NB, Best KNN

Inference: Gaussian NB’s confusion matrix visibly shows more false positives (ham misclassified as spam) than any other model, matching its lower precision figure; Best KNN shows the fewest total misclassifications of the four models.

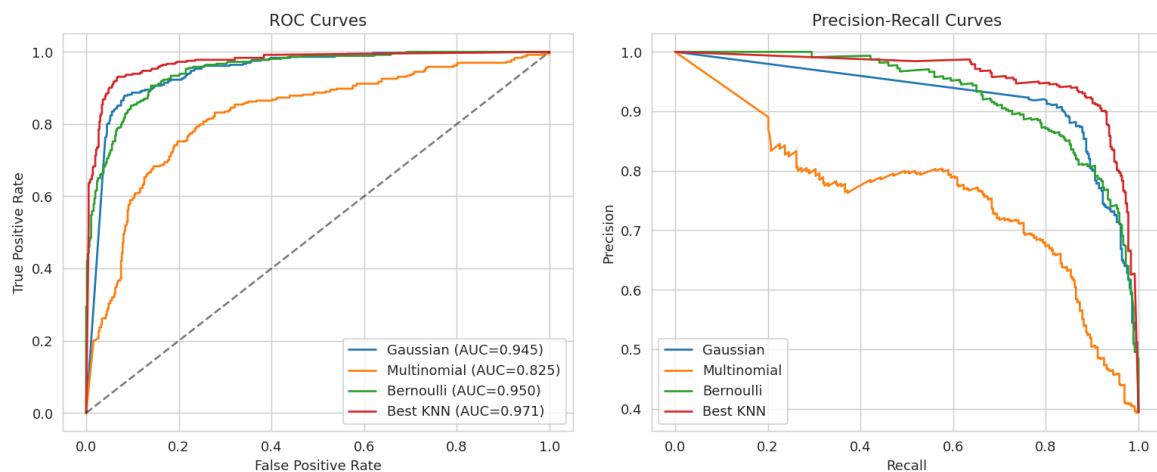


Figure 9: ROC curves and Precision-Recall curves for all four models

Inference: Best KNN’s ROC curve dominates the others across almost the entire range (highest AUC = 0.971), with Bernoulli NB close behind (0.950). Multinomial NB’s curve is visibly the weakest, consistent with its lower accuracy and F1. The precision-recall curves tell the same story more sharply, since PR curves are more sensitive to class imbalance than ROC curves.

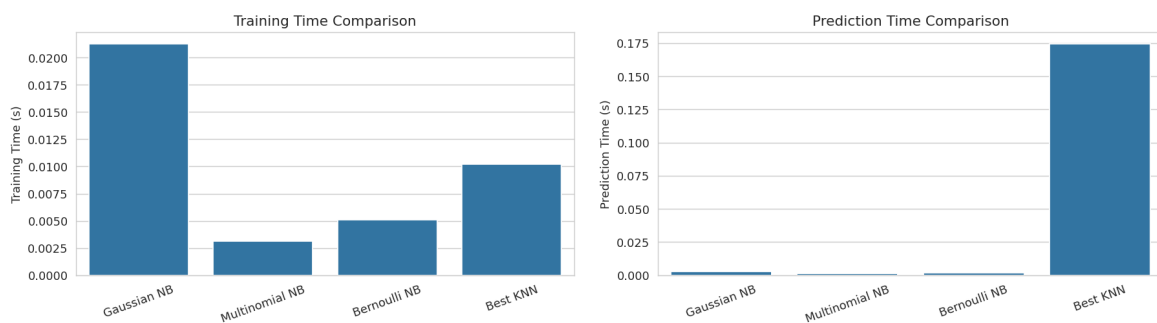


Figure 10: Training time and prediction time comparison across all four models

Inference: All three Naive Bayes variants train and predict in single-digit milliseconds, matching their theoretical $O(nd)$ training / $O(d)$ prediction cost. Best KNN trains almost as fast as Naive Bayes but is roughly 70–150 $\times$ slower at *prediction*, exactly matching the theoretical expectation that KNN prediction cost scales with n while NB prediction cost does not.

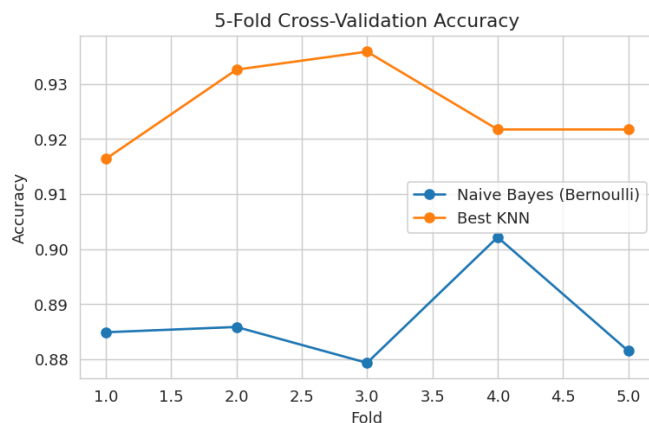


Figure 11: 5-fold cross-validation accuracy – Bernoulli NB vs. best KNN

Inference: Best KNN outperforms Bernoulli NB on every single fold, with a consistent margin of roughly 3–5 percentage points and noticeably lower fold-to-fold variance, confirming its test-set advantage is stable rather than a lucky split.

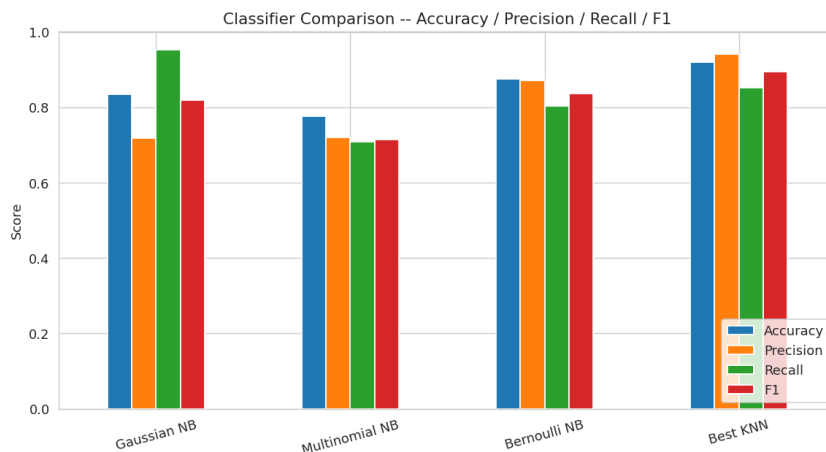


Figure 12: Classifier comparison – Accuracy / Precision / Recall / F1

Inference: Best KNN leads on Accuracy, Precision, and F1, while Gaussian NB leads only on Recall (at a real precision cost). Bernoulli NB is a clear second place overall, ahead of both Gaussian and Multinomial NB on every metric except recall.

11 Discussion

KNN, once tuned, outperforms all three Naïve Bayes variants on every classification metric for this dataset, at the cost of substantially higher prediction-time complexity – a direct, experimentally confirmed trade-off between accuracy and inference speed. Among the Naïve Bayes variants, Bernoulli NB is the clear best choice, since Spambase’s word-frequency features carry most of their discriminative signal in simple presence/absence patterns rather than exact counts or continuous distributions. KDTree and BallTree produced identical accuracy (both are exact algorithms), with BallTree marginally faster at this scale; GridSearchCV found a slightly better configuration than RandomizedSearchCV at roughly $1.75\times$ the computation time. A limitation of this experiment is

that the accuracy-vs- k curve had not yet turned downward within the tested range, so the true single-parameter optimum k (holding weights/metric fixed) may lie beyond $k = 11$; a wider sweep would clarify this. Overfitting for very small k (e.g. $k = 1$, sensitive to individual noisy points) and underfitting for very large k (predictions collapsing toward the overall class balance) are both theoretically expected, though this dataset's tested range did not clearly exhibit the underfitting regime.

12 Conclusion

For the Spambase dataset, a properly tuned KNN classifier ($k = 9$, distance-weighted, Manhattan distance) is the best-performing model overall (Accuracy 0.9207, F1 0.8944, ROC-AUC 0.9712), outperforming the best Naïve Bayes variant (Bernoulli NB) on every classification metric and on every fold of 5-fold cross-validation. This advantage comes at a real computational cost: KNN's prediction time is roughly two orders of magnitude slower than any Naïve Bayes variant, which would become the dominant practical concern for a much larger deployment dataset. Bernoulli NB is the preferred Naïve Bayes variant here, and GridSearchCV is preferred over RandomizedSearchCV when the small additional computation cost is acceptable for a marginally better hyperparameter configuration.

13 References

- [1] Scikit-learn Documentation, <https://scikit-learn.org/stable/>
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O'Reilly.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer.
- [4] Spambase Dataset, Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>